Klausur Informatik I
Sommersemester 2013
Prof. Dr. C. Damm
12. April 2013
von 08:00+x bis 09:30+x Uhr

Vorname:

Nachname:

Matrikelnummer:

Es sind **keine Hilfsmittel** (Skript, Taschenrechner etc.) erlaubt. Papier wird gestellt. Benutzen Sie nur die **ausgeteilten**, zusammengehefteten **Blätter** für Ihre Antworten. Schreiben Sie mit blauem/schwarzem Kugelschreiber, Füller etc.. Bleistift ist nicht erlaubt. **Es wird nur das gewertet, was ohne Mühe lesbar ist**. Algorithmen sind in Java-Code zu notieren, sofern keine Alternativen genannt sind.

Zum **Bestehen** der Klausur sind **45** Punkte ausreichend.

Viel Erfolg!

	Max. Punkte	Erreichte Punkte
Aufgabe 1 (Elementare Daten)	18	
Aufgabe 2 (Programmierparadigmen I)	16	
Aufgabe 3 (Strukturierte Daten)	20	
Aufgabe 4 (Programmierparadigmen II)	17	
Aufgabe 5 (Algorithmen)	19	
Summe	90	

Note:

Name: MatNr.:

Aufgabe 1 (Elementare Daten [18 Punkte])

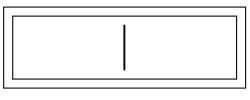
1. [6 Punkte] `a` und `b` seien `boolean-Werte`. Weisen Sie mit der Wertetafel nach, dass der Ausdruck `(!(a && b) && (a || b)) || (a && b) || !(a || b)` den Wert `true` hat (kürzen Sie bitte `true`/`false` mit 1/0 ab — der besseren Lesbarkeit wegen).

2. [6 Punkte] Sei `a` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen die vier Anweisungen?

```
System.out.println(a);
System.out.println(a+1);
System.out.println(2-a);
System.out.println(2*a);
```

3. [6 Punkte] Welche Textausgabe erzeugt folgendes Fragment?:

```
int a = 2, d = 4;
double b = 80e-1;
System.out.println( b / a );
int c = ( int ) 8.2;
System.out.println( c / a );
System.out.println( 1.0 * a / (double) d );
System.out.println( (double) a / d );
System.out.println( (double) ( a / d ) );
```



Name: MatNr.:

Aufgabe 2 (Programmierparadigmen I [16 Punkte])

1. [4 Punkte] Ein Jahr ist ein Schaltjahr, wenn es entweder durch 4 aber nicht durch 100 teilbar ist oder aber, wenn es durch 400 teilbar ist. Sei `j` eine `int`-Variable mit Wert ≥ 0. Geben Sie einen `boolean`-Ausdruck an, der genau dann den Wert `true` hat, wenn Jahr `j` ein Schaltjahr ist.

2. [5 Punkte] (A) Welche Textausgabe erzeugt das untenstehende Fragment bei der Initialisierung `int a = 14;`? (B) Formulieren Sie die Zeilen zwischen `START` und `STOP` mittels `for`-Schleife semantisch äquivalent um (d.h. unabhängig von der Initialisierung soll Ihr Fragment die gleiche Textausgabe erzeugen wie das untenstehende):

```
int a = ...; // beliebige Initialisierung mit int-Wert
  // START
{ int i = a;
    do {
        System.out.print(i);
        i-=3;
    } while ( i>=5);
} // STOP
System.out.println();
```

3. [3 Punkte]

   (A) Ergänzen Sie den Typ der Methode.

```
public static ..... square(int i) {
  System.out.println(i);
  i = i*i;}
```

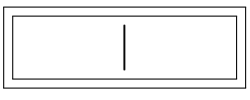
   (B) Wieviele Zeilen Textausgabe produziert die folgende Anweisung?

```
for (int i = 0; i < 5; i++) {
  System.out.println(i);
  square(i);}
```

   Begründen Sie stichpunktartig.

4. [4 Punkte] Welche Textausgabe erzeugt der Aufruf `strange(5);` ?

```
public static void strange(int n){
    if (n <=2) return;
    System.out.print(n+" ");
    strange(n-2);
    strange(n-3);
    System.out.print(n+" "); }
```



Name: MatNr.:

Aufgabe 3 (Strukturierte Daten [20 Punkte])

1. [4 Punkte] Erzeugt untenstehendes Fragment (A) einen Compilezeitfehler? Falls ja, was ist das Problem? (B) einen Laufzeitfehler? Falls ja, was ist das Problem? (C) eine Textausgabe? Falls ja, so geben Sie die Ausgabe an.

```
int[] a = {2,3,5,7,11}; int[] b = new int[6];
b = a;
a[0] = b[5];
if (a[0] % 2 == 0) System.out.println("gerade");
if (a[0] % 2 != 0) System.out.println("ungerade");
```

   Jeweils mit Ja oder Nein antworten, bei 'Ja' Fehler bzw. Ausgabe nennen:

   (A) .....;

   (B) .....;

   (C) .....;

2. [4 Punkte] `static int[][] copy2d(int[][] a)` liefert eine Referenz auf eine Inhaltskopie des übergebenen Feldes. Ergänzen Sie den Quelltext an den gekennzeichneten Stellen:

```
public static int[][] copy2d(int[][] a) {

      int n = ................;   // <-----

      int m = ................;   // <-----
      int[][] b = new int[n][m];
      for (int i = 0; i < n; i++)
          for (int j = 0; j < m; j++)
              b[i][j] = a[i][j];

      return  ................;}  // <-----
```

3. [5 Punkte] Welche Textausgabe erzeugt das folgende Codefragment?

```
int[] d = new int[10];
for (int i = 0; i < 10; i++) d[i] = 9 - i;
for (int i = 0; i < 10; i++) d[i] = d[d[i]];
for (int i = 0; i < 10; i++) System.out.print(d[i]);
System.out.println();
```

4. [3 Punkte] Welche Textausgabe erzeugt das folgende Codefragment? Begründen Sie stichpunktartig.

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b);
```

5. [4 Punkte] Ein `Zeit`-Objekt speichert eine Tageszeitangabe in Stunden und Minuten: Ist $0 \leq$ `hh` $\leq 23, 0 \leq$ `mm` ≤ 59, so liefert `jetzt = new Zeit(hh,mm);` die Referenz auf ein `Zeit`-Objekt, dass der Zeitangabe `hh:mm` entspricht. Die Methode `public void in(int h, int m)` soll „die Zeit vorstellen“ um `h` Stunden und `m` Minuten ($\mathtt{h}, \mathtt{m} \geq 0$, z.B. 12:30 um 1:20 vorgestellt ergibt 13:50). Korrigieren Sie die beiden logischen Fehler im Methodenrumpf:

```
public class Zeit {
 int hh, mm;
 public Zeit(int h, int m) { hh = h; mm = m;}
 public void in(int h, int m) {
   mm = mm + m % 60; // ZEILE 1 ENTHAELT EINEN LOGISCHEN FEHLER
   hh = hh + h % 24; // ZEILE 2 ENTHAELT EINEN LOGISCHEN FEHLER
   }
}
```

korrigierte Zeile 1:

korrigierte Zeile 2:

Name: MatNr.:

Aufgabe 4 (Programmierparadigmen II [17 Punkte])

1. [10 Punkte] Welche Textausgaben erzeugt der Aufruf `java Tier`?

```
public class Tier {
    String name, bezeichnung, laut, bewegung;
    boolean geschlecht; // true = weiblich, false = maennlich
    void laut_geben() {System.out.println(this.laut);}
    void bewegen(char a, char b) {
        System.out.print(name+" "+bewegung+" von "+a+" nach "+b + ": ");
        this.laut_geben();}
    public Tier (String nam, String bez, boolean g, String l, String bew) {
        this.name = nam; this.bezeichnung = bez;
        this.geschlecht = g; this.laut = l; this.bewegung = bew;}
    public static void main(String[] args) {
        Haustier h1 =
            new Haustier("Garfield","Kater",false,"Schnurr","schleicht");
        Haustier h2 =
            new Kuh("Lisa","Muh","geht");
        WildesTier w =
            new WildesTier("Kaa","Schlange",true,"Zisch!","schlängelt sich");
        h1.ansprache(" komm mal her!"); h1.bewegen('A','B'); // Zeilen 1, 2
        Kuh k = (Kuh) h2;
        k.ansprache("Brav "); k.bewegen('A','B');           // Zeilen 3, 4
        w.ansprache(" tu mir nichts!"); w.bewegen('B','A');} // Zeilen 5, 6
}
class Haustier extends Tier {
    void ansprache(String r) {
        String s;
        if (geschlecht) s = "Liebe "; else s = "Lieber ";
        s += this.bezeichnung; s+= " "; s += this.name;
        System.out.println(s + r);}
    public Haustier(String nam, String bez, boolean g, String l, String bew){
        super(nam,bez,g,l,bew);}
}
class Kuh extends Haustier {
    void ansprache(String s) {
        System.out.println(s + this.name + ". " + s + "!");}
    public Kuh(String nam, String l, String bew) {
        super(nam,"Kuh",true,l,bew);}
}
class WildesTier extends Tier{
    void ansprache(String r) {
        String s;
        if (geschlecht) s = "Liebe "; else s = "Lieber ";
        s += this.bezeichnung; System.out.println(s + r);}
    public WildesTier(String nam, String bez, boolean g, String l, String bew) {
        super(nam,bez,g,l,bew);}
}
```

Zeile 1:
Zeile 2:
Zeile 3:
Zeile 4:
Zeile 5:
Zeile 6:

2. [7 Punkte] Quelltexte der Klassen `Angestellter` und `Chef`:

```
public class Angestellter {
 public int alter;
 public String name;
 public Angestellter(String name, int alter) {
  this.name = name;
  this.alter = alter;}
 public String toString() {
  return name + " (" + alter + ")";}
    public static void main(String[] args) {              //
        String s, abt;                                    //
        Angestellter a = new Angestellter("Schmidt",30);  //
        Chef b = new Chef("Meier",37,"Buchhaltung");      //
          ...   // Ergaenzungen                           //
    }                                                     //
}
class Chef extends Angestellter {
 public String abteilung;
 public Chef(String name, int alter, String abteilung) {
  super(name,alter);
  this.abteilung = abteilung;}
 public String toString() {
  String s = super.toString();
  return s + " leitet " + abteilung; }
}
```

Mit welchen Fragmenten folgender Tabelle kann obige `main`-Methode ergänzt werden? Kennzeichnen Sie diese in der rechten Tabellenspalte mit "OK", die anderen mit "Compilezeit-" bzw. "Laufzeit-Fehler", je nachdem welche Probleme sie verursachen. [Die Reihenfolge der Fragmente ist irrelevant.]

Fragment	Eignung
`s = a.name;`	
`abt = a.abteilung;`	
`abt = ((Chef) a).abteilung;`	
`b = a;`	
`a = b;` `abt = a.abteilung;`	
`a = b;` `abt = ((Chef) a).abteilung;`	
`System.out.println(((Angestellter) a).toString());`	
`System.out.println(((Angestellter) b).toString());`	

Name: MatNr.:

Aufgabe 5 (Algorithmen [19 Punkte])

1. [6 Punkte] Geben Sie die Laufzeitkomplexitäten der folgenden drei Funktionen in $\mathcal{O}$-Notation an (jeweils in Abhängigkeit vom Argumentwert).

```
static int fct1(int m) {      return m;    }
static int fct2(int m) {
  int i = 0;
  while ( m > 0) {
    m = m/2;
    i++;       }
  return i;    }
static int fct3(int m) {
  int s = 0;
  for (int i = 0; i < m; i++)
  s = s + i;
  return s;    }
```

`fct1:`

`fct2:`

`fct3:`

2. [4 Punkte] Die Zahlenfolge t_n ist definiert durch die Anfangsbedingungen $t_0 = 0, t_1 = 1, t_2 = 2$ und die Rekursion $t_n = t_{n-3} + t_{n-2} + t_{n-1}$ für $n > 2$. Formulieren Sie eine iterative Java-Methode `static int t(int n)` zur Berechnung von t_n. Vervollständigen Sie dazu nur die gekennzeichneten Zeilen:

```
static int t(int n) {
    if (n <= 2) return n;
    int a = 0;
    int vvl = 0, vl = 1, l = 2;

    for (int i = 0; i < ........; i++) { // <---

        a =                             // <---

        vvl =                           // <---

        vl =                            // <---

        l =                             // <---
    }
    return a;
}
```

3. [3 Punkte] In `int[] f = {4, 6, 13, 18, 23, 100, 143, 899};` soll binär gesucht werden. Geben Sie in der Reihenfolge des Auftretens an, mit welchen Werten der Suchwert `q`=13 dabei verglichen wird.

```
static int binarysearch(int[] f, int q) {
  int a = 0, e = f.length-1, m;
  while ( a <= e) {
    m = (a+e)/2;
    if ( f[m] == q) return m;
    if (q < f[m])
      e = m-1;
    else a = m+1;}
  return -1;}
```

4. [6 Punkte] Ein Sortieralgorithmus:

```
public static void sort(int[] feld) {
    int pos = 0, tmp, n = feld.length;
    for (int i = 0; i < n; i++) {
        pos = i;
        for (int j = i+1; j < n; j++)
            if ( feld[j] < feld[pos])
                pos = j;
        tmp = feld[i]; feld[i] = feld[pos]; feld[pos] = tmp;
    }
}
```

Ergänzen Sie die Feldinhalte nach Iterationen der äußeren `for`-Schleife:

Startbelegung von `feld`	6	1	9	8	3	2	4	5	7
nach 1. Iteration									
nach 2. Iteration									
nach 3. Iteration									